Ders 5 Genişletilmiş Notu: Proje Planlama Araçları, Maliyet Tahmin Yöntemleri ve Yazılım Büyüklük Ölçümü

Ders 5 Genişletilmiş Notu: Proje Planlama Araçları, Maliyet Tahmin Yöntemleri ve Yazılım Büyüklük Ölçümü

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Zaman ve İş Planına Dönüş
- 2. Gantt Diyagramı ve İş Kırılım Yapısı
- 3. Faaliyetler Arası İlişkiler
- 4. Kritik Yol, Kilometre Taşı ve Bolluk
- 5. Kaynak Ataması
- 6. PERT Diyagramı
- 7. PERT ve CPM Farkı
- 8. Bir Proje Yönetim Yazılımında Neler Olmalı?
- 9. MS Project ile Planlama
- 10. Temel Çizgi ile Proje İzleme
- 11. Yazılım Proje Maliyet Tahmini
- 12. Bilirkişi Takdiri ve Delphi Yöntemi
- 13. Analiz Yöntemi ve Birleştirme Şaşkınlığı
- 14. İstatistik Modeller ve COCOMO
- 15. Putnam Modeli ve Büyüklük Girdisi
- 16. Yazılım Ölçümü: Doğrudan ve Dolaylı Ölçüm
- 17. İşlev Noktası Hesabı
- 18. LOC ile İşlev Noktasının Karşılaştırılması
- 19. Ders Kaynakları

1. Zaman ve İş Planına Dönüş

Yazılım geliştirme proje planını oluştururken önce risk yönetiminden söz edilmişti. Geçen hafta risk yönetimine bir kez daha dönülmüş, ardından zaman ve iş planına geçilmişti. Hoca bu hafta da aynı konudan devam etti. Bunun iki nedeni vardı. Birincisi, iş kırılım yapısının (Work Breakdown Structure, WBS) kullanımının yerleşmesi. İkincisi, Gantt ve PERT diyagramlarının üzerinde bir kez daha durulması. Bu işler MS Project ile yapıldığı için programın menü seçenekleri de derse dâhil edildi.

Dersin ikinci yarısında konu değişti. Planı çıkarılan bir projenin bütçesi nasıl tahmin edilir, bu tahmin için gereken yazılım büyüklüğü nasıl ölçülür, bu sorular işlendi.

2. Gantt Diyagramı ve İş Kırılım Yapısı

Gantt diyagramı, proje planı oluşturulurken kullanılır. Özellikle iş listesindeki her aktivitenin zaman planlaması, kaynak planlaması ve bütçelendirilmesi sırasında işe yarar. Gantt'ın önerisindeki hedef şuydu: İşlem adımlarının her birine faaliyet denir. Her faaliyetin başlangıç

ve bitiş zamanı ile bu faaliyette kullanılabilecek kaynaklar tek bir özet diyagramda gösterilir. Diyagram 1910'lardan beri kullanılmaktadır.

2.1 Kitap Satış Sitesi Örneği

Hoca, geçen haftaki örnekle devam etti: kitap satışı yapabilecek bir web sitesinin tasarlanması ve kullanıma sunulması. Bu yazılım geliştirme süreci Gantt diyagramıyla planlanacak ve izlenecek.

İlk iş, aktiviteleri genel adımlar hâlinde bloklamaktır. Örnekte şelale (waterfall) modeli kullanılıyor. Üç temel adım var: analiz, tasarım, geliştirme ve test. Geliştirme ve test birlikte adlandırılsa da birbirini takip eden işlerdir. Analiz ve tasarımın üçer alt aktivitesi vardır.

Önce WBS içinde aktiviteler, her aktivitenin kaç gün süreceği ve işin başlangıç tarihi yazılır. Örnekte iş 24 Mart'ta başlıyor ve 29 Nisan'da bitmek zorunda. Asıl soru şudur: Bu aralık en iyi şekilde nasıl planlanır?

2.2 WBS Numarası ve Gantt Sıra Numarası

WBS, alt aktiviteleri daha içeriden yazarak gösteren hiyerarşik bir iş listesidir. Her faaliyet ve alt faaliyetin kırılımı bu yapı üzerinden izlenir. Diyagramda iki ayrı numara vardır ve bunlar karıştırılmamalıdır:

- **Gantt sıra numarası:** Toplam listedeki iş numarasıdır. Aktiviteler arasındaki ilişkiler bu numarayla verilir.
- **WBS numarası:** Hiyerarşideki yeri gösterir.

Örnekte fizibilite çalışması toplamda ikinci aktivitedir. WBS'de ise birinci aktivitenin ilk alt basamağıdır, yani 1.1. Gereksinimlerin belirlenmesi toplamda üçüncü aktivitedir. WBS'de birinci ana işlemin ikinci alt başlığıdır, yani 1.2.

Bu uygulamada yalnızca ikinci düzeye inilmiştir. 1.1 vardır ama onun 1.1.1 gibi bir detayı verilmemiştir. Daha sıkı izlenmesi gereken bir projede üçüncü, hatta dördüncü seviyeye kadar alt aktiviteleri de takip etmek gerekir.

3. Faaliyetler Arası İlişkiler

Aktiviteler arasındaki ilişkiler önemlidir. Çünkü bir işin başlangıç zamanı başka bir işe bağlı olabilir. Bu bağımlılıklar öncüller (predecessors) sütununda tanımlanır.

Örnekte dördüncü aktivite, yani WBS'deki 1.3, analiz bitimi toplantısıdır. Bu toplantıda SRS son hâlini alır. Bunun için üçüncü aktivitenin, yani gereksinimlerin belirlenmesinin bitmiş olması beklenir. Aynı şekilde ayrıntılı tasarım için ön tasarımın bitmesi, ön tasarım için de analizin tamamlanması beklenir.

3.1 İlişki Tipleri

Dört ilişki tipi vardır:

Tip	Adı	Anlamı
FS	Bitiş-başlangıç (Finish to Start)	Bir işin başlaması için diğerinin bitmesi gerekir. Varsayılandır.
SS	Başlangıç-başlangıç (Start to Start)	Bir işin başlaması için diğerinin başlamış olması gerekir.
FF	Bitiş-bitiş (Finish to Finish)	Bir işin bitmesi için diğerinin de bitmiş olması gerekir.
SF	Başlangıç-bitiş (Start to Finish)	Bir işin bitmesi için diğerinin başlamış olması gerekir.

İki faaliyet arasında ilişki var dendiğinde varsayılan olarak bitiş-başlangıç anlaşılır. Bu yüzden ayrıca yazılmaz.

> **Ders sonrası not:** Hoca burada "artık buna FF yazmamız gerekmiyor" dedi; doğrusu FS'dir. Hoca hemen öncesinde varsayılanın Finish to Start olduğunu söylemişti ve yazılmayan ilişki budur.

Hoca SF için "biraz ters, örnek vermesi biraz daha zor" dedi. İlişkilere gün eklemek de mümkündür. Bağlantı kurulurken arada eklenecek bir süre varsa girilir, yoksa sıfır bırakılır.

3.2 2SS+1 Gösterimi

Gereksinimlerin belirlenmesi satırının öncül sütununda 2SS+1 yazıyor. Burada 2, fizibilite çalışmasının sıra numarasıdır. SS ilişkinin tipidir. +1 ise bir günlük eklemedir. Yani gereksinimlerin belirlenmesinin başlayabilmesi için fizibilite çalışmasının başlamış olması gerekir. Fizibilite başladıktan bir gün sonra gereksinim belirleme başlar.

> **Ders sonrası not:** Hoca önce "fizibilite çalışmasının başlayabilmesi için gereksinim belirlenmesinin" diye başladı ve aynı cümlede düzeltti. Doğrusu yukarıdaki gibidir: Önce fizibilite başlar.

3.3 Yazılmayan İlişkiler

Dördüncü aktiviteden on birinci aktiviteye kadar ilişki tipi hiç yazılmamıştır. Demek ki bunların hepsi Finish-Start ilişkisidir. Altıncı aktivitenin başlaması için dördüncünün, yedincinin başlaması için altıncının, sekizincinin başlaması için yedincinin bitmesi gerekir.

3.4 Birim Testte İki Koşul

On ikinci işlem olan birim testin iki koşulu vardır. Biri on birinci adımla bitiş-bitiş, diğeri onuncu adımla başlangıç-başlangıç ilişkisidir.

Hoca bu kararın, kullanılan yazılım geliştirme süreç modeline uygun olarak alındığını söyledi. Yorumu şöyledir:

- **FF:** Birim testlerin bitmesi için kodlama aşamasının bitmesi gerekir. Birim test bitti diyebilmek için kodlanacak yeni program kalmamış olmalıdır.
- **SS:** Birim testlerin başlaması için geliştirme aşamasının başlamış olması gerekir.

  > **Ders sonrası not:** Hoca bu kısımda önce "birim testin başlayabilmesi için 11. adımın bitmiş olması gerekiyor" ve "birim testlere başlamak için program yazımının tamamlanmış olması gerekir" dedi. Sonraki açıklamasına göre doğrusu şudur: Bitmiş olma koşulu birim testin başlamasını değil bitmesini bağlar (FF). Başlaması için kodlamanın başlamış olması yeter (SS).

4. Kritik Yol, Kilometre Taşı ve Bolluk

Kritik yol (critical path): Projenin belirlenen zamanda tamamlanması için bu yol üzerindeki tüm faaliyetlerin zamanında gerçekleşmesi gerekir. Kritik yoldaki bir faaliyet x gün ertelenirse projenin bitiş tarihi de x gün ertelenir. Bu faaliyetlere kritik faaliyetler denir. Gantt diyagramında kırmızıyla gösterilirler ve daha dikkatli davranılmaları gerekir. Örnek sıkı bir uygulamadır. Neredeyse bütün faaliyetler kritik yol üzerindedir, yalnızca bir aktivite bu yolun dışındadır.

Kilometre taşı (milestone): Proje planı için önemli olan tarihleri veya olayları belirler. Örneğin projenin bir ana bölümü bittiğinde, diğeri başlamadan önce bir değerlendirme toplantısı yapılır. Kilometre taşı Gantt diyagramında eşkenar dörtgenle gösterilir ve sıfır gün olarak belirlenir. Örnekte analiz bitimi toplantısı, tasarım bitimi toplantısı ve proje teslim toplantısı kilometre taşıdır.

Bolluk (slack): Kritik yol üzerinde bulunmayan esnek faaliyetlerin zamanlamasında yapılabilecek kayma miktarıdır. Örnekteki tek kritik olmayan iş maviyle gösterilmiştir. İşin normal bitimi belli bir noktadadır, ama bolluk süresi içinde daha geç bitmesi projeyi etkilemez.

5. Kaynak Ataması

Kaynak, proje faaliyetlerinin ilerleyebilmesi için ihtiyaç duyulan her şeydir. Yazılım projelerinde önce insan (people) gelir. Sonra ekipman gelir: kullanılacak bilgisayarlar, yönlendirme cihazları gibi ağ cihazları. Bunlar için gereken para, seyahat gibi diğer harcamalar, malzeme ve alt yükleniciler de kaynaktır. Hedef, kaynakların doğru ve etkin biçimde kullanılmasıdır.

> **Hocanın vurgusu:** En önemli kaynak insandır. İnsanların nasıl görevlendirileceği ve hangi ekip yapısında çalıştırılacağı ileride ayrıca işlenecek. Sınırlı kaynaklar projelerde öncelikli olarak planlanır, diğerleri onlara göre ayarlanır.

Proje geneli için detaylı kaynak ataması iki aşamada yapılır:

1. **Kaynak tanımı:** Kaynağın bu projede iş süresinin yüzde kaçında çalıştığı yazılır. Örnekte herkes %100 adanmış gösterilmiştir. Standart fiyat alanına her kaynak tipinin saat ücreti girilir. Hosting servisi gibi bir malzemenin ise miktarı yazılır.
2. **Atama:** Hangi kaynağın hangi aktiviteye ne kadar süreyle atandığı belirtilir.

6. PERT Diyagramı

Gantt ile birlikte kullanılan diğer diyagram PERT'tir (Program Evaluation and Review Technique). Graf temelli bir yöntemdir. Projedeki faaliyetler düğümlerle, faaliyetler arasındaki bağlantılar kenarlarla gösterilir.

İkisinin güçlü yanları farklıdır. Gantt çizelgesi faaliyetlerin paralelliğini göstermekte başarılıdır. Ancak hangi faaliyetin hangi faaliyetlere dolaylı olarak bağlandığını göstermekte yetersiz kalır. PERT bu boşluğu doldurur.

MS Project'in ağ diyagramı (network diagram) seçeneği, Gantt çizelgesini PERT olarak ifade eder. Bu görünümde şunlar okunur:

- Kırmızı kutular kritik görevlerdir.
- Her dikdörtgen bir faaliyettir.
- Gantt'ta eşkenar dörtgenle gösterilen kilometre taşları burada da görünür.
- Bir faaliyet kutusunda başlangıç tarihi, bitiş tarihi, süre ve atanan kaynak bulunur. Örneğin fizibilite çalışmasının süresi 4 gündür ve kaynağı bir analisttir. Ön tasarım ve ayrıntılı tasarım için de süreler, örneğin 3 gün, tanımlanır.

Buradaki hedef, kritik yolu PERT ile de bulmak ve faaliyetlerin birbiriyle en iyi nasıl ilişkilendirileceğini görmektir. Farklı yollarla sonuca nasıl ulaşılabileceği de bu diyagramda ayrıntılı olarak görülür.

7. PERT ve CPM Farkı

PERT'in alternatifi Kritik Yol Yöntemi'dir (Critical Path Method, CPM). Hoca bu farkın slaytta yazılı olmadığını söyleyip kendisi ekledi. İkisi de yöneylem araştırmasında şebeke (network) diyagramları diye geçer. Çünkü faaliyetlerin nasıl bir ağ oluşturduğunu, başlangıçtan bitişe kadar arada hangi faaliyetler olduğunu ve birbirinin yerini tutabilecek faaliyetler bulunup bulunmadığını gösterirler.

Fark şudur: PERT süreleri belirsiz aktiviteler için, CPM süreleri önceden belirli aktiviteler için kullanılır.

CPM'e örnek bir otomasyon sürecidir. Her prosesin içindeki her faaliyetin süresi dakikasına kadar bellidir ve bu süre bir uygulamadan diğerine değişmez. Böyle bir belirlilik varsa şebeke diyagramı CPM ile çözülebilir.

Yazılım geliştirmede ise durum böyle değildir. Analize ve onun altındaki fizibilite ile gereksinim belirlemeye farklı uygulamalarda farklı süreler gerekebilir. Süreler projenin cinsine, müşterinin tipine ve uygulama alanına göre değişir.

> **Ders sonrası not:** Hocanın "CPM PERT'den daha karmaşık CPM'e göre" cümlesi altyazıda karışıktır. Büyük olasılıkla kastedilen, PERT'in CPM'e göre daha karmaşık olduğudur. Çünkü PERT üç süre tahmini kullanır, CPM ise önceden belli tek süreyi kullanır.

7.1 PERT'te Süre Hesabı

Diyagramdaki fizibilite süresi 4 gündür. Ama bu değer hazır verilmez, bulunması gerekir. Belirsizlik olduğu için bu uygulamaya özgü üç süre tahmin edilir: en iyimser (t_o), en olası (t_m) ve en kötümser (t_p). Beklenen süre şöyle hesaplanır:

$$t_e = \frac{t_o + 4t_m + t_p}{6}$$

> **Ders sonrası not:** Hoca formülü "en kötümser süre + 4 × en olası süre + en kötümser süre, bölü 6" diye okudu; doğrusu ilk terimin iyimser süre olmasıdır. Hoca hemen öncesinde üç ayrı süre saymıştı.

Hoca bu hesabın paket programla yapılacağını da söyledi: "PERT'i siz yazılım kullanarak oluşturursunuz, en olası süreleri size istatistiksel hesaplamalarla verir."

> **Ders sonrası not:** Program üç süre tahminini kendiliğinden üretmez. Büyük olasılıkla kastedilen, girilen tahminlerden beklenen sürenin programca hesaplanmasıdır.

Hoca ardından sınıfa farkı sordu.

Soru: PERT ile CPM arasındaki fark nedir? **Cevap:** Bir öğrenci, PERT'in süreleri belirsiz aktiviteler için, Critical Path Method'un süreleri kesin tanımlı aktiviteler için kullanıldığını söyledi. Hoca cevabı doğruladı.

8. Bir Proje Yönetim Yazılımında Neler Olmalı?

Proje izlemeyle ilgili çok sayıda yazılım vardır. Yazılım geliştirme süreç modellerine daha uygun başka proje yönetim araçları da tanıtılmıştır. Hoca yalnızca proje planını izlemek için en yaygın olanı gösterdi. Slaytı MS Project'in eski bir sürümünün arayüzüyle hazırlamıştı, önemli olan hangi seçeneklerin bulunduğuydu. MS Project ya da başka bir proje yönetim yazılımı en az şunları yapabilmelidir:

- Faaliyetleri takvim üzerinde göstermek ve Gantt şeması çizmek.
- PERT diyagramı çizmek.
- Faaliyette kullanılan kaynakları göstermek ve kaynak kullanım yüzdelerini analiz etmek.
- Kaynak maliyetlerini analiz etmek.

Maliyet analizi şöyle işler: Gantt'taki kaynak atamaları ve süreler, kaynak tanımındaki saatlik veya aylık ücretlerle birleştirilir. Böylece bütçede hangi kaynağın ne kadar kullanıldığı görülür.

Gantt diyagramından şu çıktılar alınır:

- Faaliyetlerin süresi, başlangıç ve bitiş tarihleri.
- Faaliyetlerin birbirine bağımlılıkları.
- Kritik yol üzerinde olup olmadıkları.
- Kritik yolda olmayan faaliyetlerin bolluk süreleri.
- Proje başlama tarihi belirlendiğinde projenin toplam süresi ve bitiş tarihi.

Bulunan bitiş tarihi bize verilen süreyle eşleşmiyorsa proje toparlanır, yani sıkıştırılarak yeniden planlanır.

9. MS Project ile Planlama

Planlama şu sırayla ilerler:

1. Projenin başlangıç tarihi ayarlanır.
2. Görevler, yani faaliyetler girilir.
3. Her faaliyetin ne kadar süreceği hesaplanır ve girilir.
4. Kaynak listesi hazırlanır, kaynak ataması yapılır.
5. Görevler arasındaki bağlantılar kurulur.
6. Plana son bir ayar yapılır ve temel çizgi (baseline) oluşturulur.

9.1 Görev Türleri ve Bağlantılar

Görev türü farklı olabilir ve her görev için belirlenir. Slaytta "yapılacak iş miktarı sabittir" ve "kaynak ve iş değişikliği görev süresini etkilemez" gibi seçenekler geçiyor.

> **Ek bilgi (derste anlatılmadı):** MS Project'te büyük olasılıkla üç görev türü vardır: sabit birim (fixed units), sabit iş (fixed work) ve sabit süre (fixed duration). Bu tür, kaynak sayısı veya iş miktarı değiştiğinde programın hangi değeri koruyup hangisini yeniden hesaplayacağını belirler. Örneğin sabit sürede kaynak eklemek süreyi kısaltmaz.

Bağlantı kurulurken iki şey seçilir: ilişkinin hangi faaliyetle kurulduğu ve araya süre eklenip eklenmeyeceği. Birden fazla görev finish-start ile bağlanabilir. Çoğunlukla varsayılan olan Finish-Start kullanılır. Bir faaliyet için bolluk süresi varsa, normal bitişiyle bitebileceği son nokta arasındaki aralık diyagramda görünür. Bolluk hiç olmayabilir de.

> **Ders sonrası not:** Hoca SS için "birinci görevin başlaması için diğer iki görevin başlamış olması" dedi. Bu ifade altyazıda karışıktır; büyük olasılıkla kastedilen, bağlı görevin başlaması için diğer görevin başlamış olmasıdır. SS tanımı Bölüm 3.1'deki gibidir.

Kritik yolda ileriye doğru gidişteki süreler ile geriye doğru gelişteki süreler eşittir, arada farklılık yoktur.

> **Ek bilgi (derste anlatılmadı):** İleri geçişte her faaliyetin en erken başlama ve bitiş zamanları (ES, EF), geri geçişte en geç başlama ve bitiş zamanları (LS, LF) bulunur. Bolluk $LS - ES$ farkıdır. Kritik yoldaki faaliyetlerde iki zaman çakışır, bu yüzden bolluk sıfırdır.

Bir aktiviteyi birden fazla kişi yapabilir. Kaynak planlama aşamasında önce görev tipi yazılır: "iki sistem analisti çalışacak" gibi. Atama gerçekleşince bu özelleştirilir. "Sistem analisti 1" ve "sistem analisti 2" yerine doğrudan kişilerin adları yazılır.

9.2 Proje Bilgisi

Proje bilgisi menüsünde en başta şunlar belirlenir: projenin başlangıç tarihi, öncelik derecesi, takvim türü ve tatil günleri. Ayrıca hangi para biriminin kullanılacağı ve görevlerin saatle mi, sabit birimlerle mi girileceği de burada verilir. Bu açıklamalar hem iş listesiyle hem projenin tipiyle ilgilidir.

9.3 Takvim Ayarları

Araçlar (Tools) seçeneğinde takvimin haftanın hangi günüyle başlayacağı belirlenir. Ayrıca klasik ocakta başlayan takvimin mi, başka bir takvimin mi kullanılacağı seçilir. Hocanın örneği, mali yılın mart ayında başlamasıydı.

Çalışma süreleri de burada tanımlanır ve her görev bu zamana göre ayarlanır. Örnekteki değerler şunlardır: çalışma saatleri 8-17, günde 8 saat, haftada 40 saat, hafta sonu iki gün tatil, ayda 20 iş günü. Bayram gibi projenin içine giren resmî tatiller de takvim kısmında belirtilir.

9.4 Çalışma Grubunun Bilgilendirilmesi

Bir de iletişim yeteneği gerekir. İş grubu belirlenir. Ekibe iş atamaları ve olaylar duyurulur, hangi kişilere hangi görevlerin atandığı iletilir. Hoca, son dönemde Teams uygulamalarıyla bunun çok gelişmiş örneklerinin bulunduğunu söyledi. Bu yetenek özellikle çok uluslu ekiplerde veya birkaç şirketin birlikte çalıştığı yapılarda önem kazanır.

10. Temel Çizgi ile Proje İzleme

Temel çizgi (baseline), yapılandırma yönetiminde (configuration management) de geçen bir kavramdır. Proje planı tamamlanınca o andaki plan ilk plandır ve kaydedilir. Sonra proje başlar ve izleme aşamasına geçilir.

Sürüm tutmak gerekir, çünkü plan uygulandıkça gerçek ilerleme ortaya çıkar. Her aşamadaki planın durumu bir temel çizgiyle arşive kaydedilir. Gerçekleşen durum planlanan durumla kıyaslanır. Temel çizginin hangi zamana ait olduğu da belirtilir: analiz sonu, tasarım sonu gibi. Örneğin analiz bitimindeki toplantıya göre, başarılı olmak için planın nasıl değişmesi gerektiği ortaya konur.

Döngü şöyle işler:

1. Dönüm noktasında planı ve projenin durumunu temel çizgi olarak kaydet.
2. Gerçekleşenleri gir.
3. İlk planda tanımlananla gerçekleşeni karşılaştır.
4. Normal planla gitmekte zorluk varsa planı güncelle.
5. Bütün temel aşamalar tamamlanana kadar döngüyü sürdür.
6. Projeyi kapat ve projeyle ilgili saklanacak bilgileri arşive kaydet.

10.1 İzleme Sırasında Girilen Bilgiler

Gerçekleşen kaynak bilgileri için her kaynağın standart ücreti ve fazla mesai ücreti ücretler kısmında tanımlanır.

Atama bilgisi için ise şunlar girilir: işin ne kadarının tamamlandığı, kalan iş için gereken süre, gerçek başlama tarihi, iş bitmişse bitirme tarihi, proje grubuna kimlerin atandığı ve başlangıçtan bitişe kadarki maliyet. Örneğin analiz yapan analistlerin işin ne kadarını bitirdiği girilir. Kaynak bilgisiyle birleşince maliyet oranı bir tabloda görülebilir. Kalan işin fazla mesaiyle mi, normal mesaiyle mi bitirileceği de belirtilmelidir.

Görevler filtrelenebilir: bütün görevlere göre, tamamlanmış görevlere göre, tamamlanmamış görevlere göre veya kritik görevlere göre. Bu, iş listesindeki bilgiye ulaşmayı kolaylaştırır.

10.2 Araç Seçimi ve Uygulamadan Bir Örnek

Hoca, MS Project dışında ekip çalışmasını daha iyi destekleyen ve daha yetenekli pek çok proje yönetim yazılımı bulunduğunu söyledi. Açık kaynaklı programlar da vardır. Asgari çerçevede en az MS Project kadar bir aracın kullanılabilmesi gerekir. Daha önce kullanmamış olanlara önerisi küçük bir örnekle denemeye başlamaktır.

Soru: Hiç proje yönetim yazılımı kullanmayan var mı? Denetlerken süreler diyagram üzerinden takip edildi mi? **Cevap:** Projeleri denetim tarafından gören bir öğrenci, bu araçları kendisinin birebir kullanmadığını söyledi. Denetimde şunlar kontrol ediliyormuş: tasarım dokümanı hazırlanmış mı, sorumlular belirlenmiş mi, maliyet ve süre analizleri tanımlanmış mı, testler yapılmış mı. Yani projede olması gereken ayrıntıların belli bir standartta ilerleyip ilerlemediğine bakılıyor. Hoca bu durumda küçük bir örnekle başlamayı önerdi.

11. Yazılım Proje Maliyet Tahmini

Aradan sonra hoca şimdiye kadar yapılanları özetledi. Aktiviteler Gantt ve PERT ile seviyelendirildi. Her faaliyetin proje sürecindeki yeri ve başka faaliyetleri nasıl etkilediği görüldü. Kaynaklar önce görev, sonra kişi olarak atandı. Ama bir eksik kaldı: Bu projenin bütçesi çıkarılmalı, maliyeti mümkün olduğunca doğru saptanmalıdır.

Planlama aşamasındaki tahminlerin tutarlı ve güvenli olmadığı, sonradan maliyet sorunları çıktığı görülmektedir. Analiz ve tasarım aşamalarında plan güncellendikçe tahmin iyiye doğru götürülür. Bu bölümün sorusu şudur: Planlama aşamasında nasıl bir yol izlenirse tahmin daha güvenilir olur?

Yıllardır kullanılan yöntemler dört başlıkta toplanır:

- **Bilirkişi takdiri:** Kurumda benzer projelerde çalışmış deneyimli proje yöneticisi, bitmiş projelere benzeterek kafasındaki arşive göre yeni projeye değer biçer.
- **Delphi yöntemi:** Birden fazla uzmanın fikirleri birleştirilir, bir anlaşma noktasında değer tahmin edilir.
- **Analiz yöntemi:** Büyük proje aşamalarına veya alt modüllerine ayrılır, tahmin her parça için yapılır.

- **İstatistik modeller:** Uzmanın kafasındaki birikim yerine bitmiş projeler arşivi kullanılır. Arşivde hangi tür projenin ne kadar sürede ve ne kadar maliyetle tamamlandığı, hangi aşamanın ne kadara mal olduğu ve risklerin neler olduğu bulunur.

12. Bilirkişi Takdiri ve Delphi Yöntemi

Bilirkişi takdiri eski yıllardan beri kullanılır. Bilirkişi, yazılım birimindeki en kıdemli, en çok projede çalışmış, proje yönetimi ve bütçeleme kararları da vermiş uzmandır. Bu uzman analiz yöntemini de kullanabilir.

Delphi yöntemi şu adımlarla işler:

1. Sistem tanımı yapılır.
2. Uzmanlık gerektiren bilirkişiler seçilir.
3. Maliyet konusunda anket soruları hazırlanır ve dağıtılır.
4. Anket sonuçları analiz edilir ve ortak karara yakın olup olmadığına bakılır.
5. Karar yakınsa rapor hazırlanır ve maliyet tahmini bildirilir.
6. Bildirilen maliyetler birbirinden çok farklıysa istenen bilgiler verilir, yanıtlar düzenlenir ve anket yeniden dağıtılır. Sonra dördüncü adıma dönülür.

Delphi yalnızca maliyet tahmininde kullanılmaz. Ortak karara varmak için kullanılan genel bir yöntemin adıdır. Burada tek bir bilirkişi yerine birkaç bilirkişi bir araya getirilerek yeni projenin maliyeti planlama aşamasında tahmin edilir.

13. Analiz Yöntemi ve Birleştirme Şaşkınlığı

Analiz yöntemi bilirkişi tahminine, Delphi'ye, hatta istatistik modellere de uygulanabilir. Ürün veya işlem hiyerarşik olarak elemanlara ayrılır. En alt elemandan başlanır ve yukarı doğru her öğe için gider takdir edilir. Bunun için önce ürüne veya üretime dayalı bir analiz kartı düzenlenir. İki şekilde yapılır:

- **Ürüne göre analiz:** Yazılımın alt modülleri belirlenir. Eski proje bilgileri göz önünde tutularak her modülün ne kadar sürede tamamlanacağı, kaç kişinin çalışacağı ve sonuçta maliyetin ne olacağı bulunur.
- **Üretim aşamasına göre analiz:** Her evrede kaç kişinin, kaç uzmanın gerektiği ve bunların maliyeti bulunur. Hoca aşamalar için iki örnek verdi:
  - Proje planlama, analiz, tasarım, kodlama, test.
  - Proje yönetimi, geliştirme (tasarım, kodlama, birim test), test (birleştirme ve kabul testi).

13.1 Birleştirme Şaşkınlığı

Analiz yönteminde ilk denemede doğruyu bulmak mümkün değildir, üzerinde çalışmak gerekir. Hoca bunu iki örnekle açtı.

Ürüne göre gidelim. Yazılımın 5-6 modülü olsun. Her modül için kimlerin çalışacağı belirlenir: aynı anda çalışan ekip mi, sırayla çalışan ekip mi? Sonra toplam iş gücü harcaması bulunur ve bunun elimizdeki iş gücünü ne kadar aştığına bakılır.

Üretime göre gidelim. Analiz aşamasında bilgi toplama, modelleme ve SRS raporu hazırlama için kimlerin ne kadar süre çalışacağı belirlenir. Bu, Gantt'taki kaynak atamasının aynısıdır. En alt işlemden yukarı çıkıldığında analiz aşamasının toplamında ne kadar analist ve ne kadar proje yöneticisi gerektiği ortaya çıkar.

Bu toplam alınınca birleştirme şaşkınlığı denen durumla karşılaşılabilir. Her alt işleme yeterince zaman verelim, hiçbir yerde hata çıkmasın diye düşünülür. Parçalar birleştirilince, bütün projeye yetmesi gereken iş gücünün yarısı yalnızca analizde harcanmış olur. O zaman geri dönülür, işler sıkıştırılır, analistlere daha fazla yük verilir ve zaman planlaması yeniden yapılır. Kaynak planlaması da bununla birlikte yenilenir.

> **Hocanın vurgusu:** Analiz yöntemi bu açıdan iyidir. Her şeyi ideal yapalım diye çok bol süreler verildiğinde asıl sürenin ve asıl maliyetin üstüne çıkılmasını engeller. Yeniden çalışılır ve yeni süreler belirlenir.

14. İstatistik Modeller ve COCOMO

Yöntemlerin içinde en çok veriye dayananlar yazılım maliyet tahmin modelleridir. Bunlar birer istatistik modeldir. Doğrudan ölçülemeyen bir Y değeri, bağımsız bir değişken yardımıyla kestirilir: $Y = f(X)$. Yazılım projelerinin tahmininde kullanılan fonksiyon şudur:

$$Y = aX^b$$

Bu modeller analiz yöntemiyle birlikte kullanılır.

14.1 Temel COCOMO

COCOMO (Constructive Cost Model), yazılım projesi hesabı için bir tahmin modelidir. Hoca önce temel modeli anlamayı, sonra geliştirmeyi önerdi. Boehm modeli ilk kullandığında iş hacmini bin kod satırına dayanarak kestirmişti. LOC (line of code) kod satırı sayısını, KLOC bin kod satırını gösterir. Soru şudur: Bu kadar bin satırlık bir yazılım ne kadar iş hacmiyle geliştirilir?

Satır sayısı çok objektif bir ölçü değildir. Yazılım geliştiren herkes bunu bilir. Ama doğrudan ve kolay ölçüldüğü için yine de kullanılabilir.

Genel formül burada iş hacmi (H) için şöyle yazılır:

$$H = a \cdot \text{KLOC}^b$$

Süre ise bu kadar iş gücünün ne kadar süreye yayılırsa anlamlı olacağının karşılığıdır:

$$T = c \cdot H^d$$

a, b ve c, d katsayıları, çok sayıda proje üzerinde çalışılarak bulunan en iyi değerlerdir. Hocanın söylediğine göre COCOMO hesaplanırken 64 proje üzerinde çalışılmış, sonuçlardan istatistik yöntemlerle a ve b'nin hangi durumda ne olması gerektiği bulunmuştur.

Slaytta örnek katsayılar da vardı. Hoca bunları "3.6 ve üstü B değeri, 3.6 ile bu 20 bizim katsayılarımız" diye okudu.

> **Ders sonrası not:** Hoca 64 proje dedi; Boehm'in özgün çalışması büyük olasılıkla 63 projeye dayanır. Hocanın iş hacmi için kullandığı H harfi literatürde çoğunlukla E ile gösterilir. Hoca Boehm için "yazılım mühendisliğinin kurucusu" dedi; büyük olasılıkla "kurucularından biri" demek daha doğrudur.
>
> **Ek bilgi (derste anlatılmadı):** Temel COCOMO'da katsayılar proje türüne göre değişir. Klasik tabloda büyük olasılıkla basit (organic) proje için $a = 2{,}4,\ b = 1{,}05$, yarı ayrık (semi-detached) proje için $a = 3{,}0,\ b = 1{,}12$, gömülü (embedded) proje için $a = 3{,}6,\ b = 1{,}20$ alınır. Bu yüzden slayttaki değer büyük olasılıkla gömülü türe aittir.

14.2 Orta COCOMO

Orta COCOMO modeli biraz daha hassaslaştırır. Ürün, donanım, iş gücü ve proje özelliklerinden oluşan bir tablodan bir düzeltme faktörü (EAF) gelir ve formüle eklenir:

$$H = a \cdot \text{KLOC}^b \cdot \text{EAF}$$

Hoca düzeltme faktörünün 0,90 ile 1,40 arasında alındığını söyledi. Hangi tür yazılımın hangi faktörle alınacağı hesaplanabilir. Yalnızca temel COCOMO kullanılırsa bu faktör olmaz.

14.3 COCOMO'nun Uygulanması

Hoca Boehm'i yazılım ekonomisinde ilk çalışmaları yapan uzman olarak tanıttı. Modelin uygulanışı şöyledir:

1. Üründeki bütün alt sistem ve modüller tanımlanır.
2. Her modülün büyüklüğü tahmin edilir. Bu tahminlerden tamamının büyüklüğü bulunur.
3. Modül düzeyinde ürün, donanım, personel ve uygulanan teknoloji özelliklerine göre düzeltme faktörleri kullanılır.
4. Her modül için iş hacmi ve proje süresi hesaplanır.
5. Analiz yöntemi kullanıldığı için değerler yukarı doğru toplanır. Toplam iş gücü ve proje süresi bulunur.
6. Fayda-yarar değerlendirmesi için duyarlılık analizi yapılır.
7. Yalnızca tasarım değil, planlama ve analiz giderleri de hesaplanır.
8. Sonuçlar Delphi yöntemiyle birkaç uzman tarafından değerlendirilir ve kullanıma alınır.

15. Putnam Modeli ve Büyüklük Girdisi

COCOMO dışında da modeller vardır. Hoca Putnam yöntemini başka bir istatistik tahmin yöntemi olarak gösterdi. Bu modelde iş gücü (H); kod sayısı (KLOC), teknoloji katsayısı (C_K) ve yıl cinsinden geliştirme süresine bağlıdır. Ne kadar sürede geliştirileceği tahmini de hesaba girer.

> **Ders sonrası not:** Altyazıdaki "Putnam Reyle/Reili" büyük olasılıkla Putnam'ın Rayleigh eğrisine dayanan modelidir.

Modellerin dayanma noktası büyüklüktür. Temel COCOMO'da iş gücünü bulmak için projenin ne kadar büyük olduğu sorulur. İki modelde de başlangıç noktası olarak KLOC kullanılmıştır. Ama satır sayısı tek büyüklük ölçütü değildir, projenin büyüklüğünü yansıtmak için işlev noktası (Function Point) değeri de kullanılabilir.

Yeni başlayacak bir projenin tahmini olarak kaç işlev noktası veya kaç satır olacağı çıkış noktasıdır. Bundan sonra şu sorular cevaplanır: Bu büyüklükte bir proje için ne kadar iş gücü ve süre gerekir? Bu iş gücü ne kadar süreye yayılmalıdır?

16. Yazılım Ölçümü: Doğrudan ve Dolaylı Ölçüm

Mühendislik yaklaşımıyla yazılımın bütün adımlarında ölçme yapılır. Yazılımın karmaşıklık ölçütü, döngüsel karmaşıklık (cyclomatic complexity) ölçütü gibi çok sayıda ölçüt vardır. Bu derste yalnızca büyüklük ölçümü işlendi. Büyüklük ölçümü doğrudan veya dolaylı olabilir.

Doğrudan ölçüm: Gider, süre, komut satırı ve hız doğrudan ölçülür. En önemli örneği satır sayısıdır.

Dolaylı ölçüm: Tek bir değere değil, birden fazla değere bakılır. Örneği işlev noktasıdır.

LOC ve KLOC değerleri şu alanlarda fayda sağlar:

- **Verimlilik:** Bu büyüklükte bir yazılım için kaç kişi kaç ay çalıştı?
- **Kalite:** Bu kadar kod satırında belirli bir kullanım süresinde kaç hata çıktı?
- **Gider:** Bu büyüklükteki kodu yazmanın toplam gideri ne oldu?
- **Belgeleme:** Ne kadar belge hazırlandı?

17. İşlev Noktası Hesabı

İşlev noktası (Function Point, FP) ortaya atıldığında belirlenmiş ölçütler vardır. Bunlar satırlara yazılır ve sayılır. Sayım kolay değildir, ama yazılımda sayarak gidilir.

17.1 Beş Ölçüt ve Ağırlık Faktörleri

Yeni başlayacak bir yazılım için şunlar kestirilir:

- Kullanıcının yazılıma giriş (girdi) sayısı.
- Kullanıcının aldığı çıktı sayısı.

- Kullanıcının sorgulama sayısı.
- Dosya sayısı.
- Dış arabirim sayısı.

Ağırlık faktörü, ölçütün diğerlerine göre önemini yansıtır. Her ölçüt basit, orta veya karmaşık sayılır. Hocanın okuduğu değerler şunlardır:

Ölçüt	Basit	Orta	Karmaşık
Girdi sayısı	3	4	6
Dosya sayısı	7	10	15

> **Ders sonrası not:** Hoca dosya için "dosya sayısı mesela 7 ise basit" ve "dosya sayısı çok fazla olduğunda 15 etkiliyor" dedi. Doğrusu 7, 10 ve 15'in dosya sayısı değil ağırlık olmasıdır. Hoca da hemen sonra sayının ağırlıkla çarpıldığını söyledi.
>
> **Ek bilgi (derste anlatılmadı):** Klasik işlev noktası tablosunda diğer ağırlıklar büyük olasılıkla şöyledir: çıktı 4/5/7, sorgu 3/4/6, dış arabirim 5/7/10. Bu değerler büyük olasılıkla Albrecht'in yöntemine dayanır ve Pressman'ın kitabında da verilir.

Her satırda sayı, seçilen ağırlıkla çarpılır. Örneğin girdi sayısı basit grubuna giriyorsa 3 ile, orta grubuna giriyorsa 4 ile çarpılır. Hangi grubun alınacağı için tablolar vardır. Yazılımın basit, orta ya da karmaşık oluşu tahmin yoluyla kestirilir. Satır çarpımlarının toplamına **sayı toplamı** denir.

17.2 Düzeltme Formülü

Sayı toplamının üzerinde bir düzeltme işlemi yapılır:

$$FP = \text{sayı toplamı} \times \left(0{,}65 + 0{,}01 \sum_{i=1}^{14} F_i \right)$$

Buradaki F_i değerleri, 14 soruya verilen cevaplardır.

17.3 On Dört Genel Özellik Sorusu

Yukarıdaki beş ölçüt daha çok ticari uygulamaları anımsatır. Yalnız bunlarla karar vermemek için yazılımın genel özellikleriyle ilgili 14 soru eklenir. Geliştiriciler her soruya 0 ile 5 arasında değer verir. Slayttaki değerler örnektir, sabit değildir. Hocanın irdelediği sorular şunlardır:

- **Güvenli yedekleme ve geri yükleme gerekli mi?** Çok önemliyse 4 ya da 5, daha az önemliyse 3 ya da 2 verilir. Hocaya göre bu aslında her yazılımda önemlidir.
- **İletişim altyapısı gerekli mi?** Program çevrim içi mi çalışıyor? Tek başına çalışan (standalone) bir programsa 2 verilebilir. Günümüzde yazılımların çoğu çevrim içi çalıştığı için çoğunlukla 4 ya da 5 olur.
- **Dağıtılmış işleme fonksiyonları var mı?** Yazılım birçok farklı şubede çalışır durumda mı?
- **Performans kritik mi?**
- **Sistem yükü fazla mı?**
- **Çevrim içi veri girişi var mı?**

17.4 Slayttaki Örnek

Hoca sayı toplamının değerini okumadı, yalnızca "büyük bir sayı" dedi. Düzeltme kısmı için slayttaki toplam verildi.

- Verilen: $\sum_{i=1}^{14} F_i = 32$.
- İşlem: $0{,}65 + 0{,}01 \times 32$.
- Ara sonuç: $0{,}01 \times 32 = 0{,}32$ (ara adım eklendi).
- Sonuç: $0{,}65 + 0{,}32 = 0{,}97$.

Yani bu örnekte işlev noktası, sayı toplamının 0,97 katıdır. Sayı toplamı biraz küçülür.

> **Ders sonrası not:** Hoca "0,65 + 0,01 toplam F_i ile toplayacağız" dedi; doğrusu sayı toplamının bu değerle çarpılmasıdır. Hocanın kendisi de formülü önce "çarpı" diye okumuştu. Hocanın "sayı toplamını küçültmüş olacak" sözü bu örnek için doğrudur, ama genel kural değildir. 14 sorunun her biri en çok 5 aldığı için $\sum F_i$ en fazla 70 olur ve çarpan 0,65 ile 1,35 arasında değişir. $\sum F_i > 35$ olduğunda çarpan 1'i aşar, sayı toplamı büyür.

17.5 İşlev Noktasının Kullanım Alanları

LOC ve KLOC nasıl verimlilik, kalite, gider ve belgelemede kullanılıyorsa, tahmin yoluyla bulunan bu büyüklük değeri de aynı alanlarda kullanılır. Proje sonlandırıldığında bilgileri arşive saklanır. Her bitmiş proje için işlev noktası büyüklüğü belirlendikten sonra şunlar söylenebilir:

- **Verimlilik:** Kaç kişi kaç ay çalıştı?
- **Kalite:** Kullanım başladıktan sonra bir ay boyunca kaç hata çıktı?
- **Gider:** Toplam gider ne kadar, her işlev noktası başına gider ne kadar?
- **Belgeleme:** Her işlev noktası başına kaç sayfa belge hazırlandı?

18. LOC ile İşlev Noktasının Karşılaştırılması

Ölçüt	LOC	İşlev noktası
Programlama dili	Bağlıdır	Bağımsızdır
Ölçünün niteliği	Doğrudan ölçü	Daha öznel, doğrudan fiziksel ölçü değildir, kullanım özelliklerine göre verilir
Veri toplama	Satır sayılır	Daha güçtür: girdi, çıktı, veri tabanı, dosya sayısı gibi alt başlıklara inilir
Zayıf yön	İyi tasarlanmış ama kısa yazılımları değerlendirmek zordur; işlemsel olmayan dillere kolay uyarlanmaz	Tahmine dayanır

Hoca, maliyet tahminleri slaytını tersten kullandıklarını da belirtti. Önce maliyet tahminleri, sonra bunlar için gerekli büyüklük değerleri konuşuldu.

19. Ders Kaynakları

Hoca proje yönetimi konuları için iki kitap gösterdi. Pressman'ın *Software Engineering* kitabındaki proje yönetimi konuları çok geniştir. Diğeri *Information Systems Project Management* adlı, AMACOM yayınlarından çıkan kitaptır.

Dersin Toparlaması

Dersin ilk yarısında proje planı araçlar üzerinden yeniden kuruldu. Gantt ve WBS, faaliyetleri ve bağımlılıkları gösterir. Kritik yol, kilometre taşı ve bolluk bu planın okunmasını sağlar. PERT, süreleri belirsiz faaliyetlerde üç tahminden beklenen süreyi bulur ve bu yüzden yazılım projelerine uyar. MS Project gibi araçlar planı kurmanın yanında temel çizgilerle plan-gerçekleşen karşılaştırması yapar.

İkinci yarıda bu planın bütçesine geçildi. Bilirkişi takdiri ve Delphi uzman görüşüne, istatistik modeller bitmiş proje arşivine dayanır. Analiz yöntemi hepsiyle birlikte kullanılabilir ve bol süre vermenin sonucunu birleştirme şaşkınlığıyla açığa çıkarır. COCOMO ve Putnam gibi modeller bir büyüklük girdisi ister. Bu girdi ya doğrudan ölçülen LOC ya da dolaylı ölçülen işlev noktasıdır. Hoca ayrıca sonraki bölümlerde insan kaynağının görevlendirilmesi ve ekip yapısı üzerinde duracağını söylemişti.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.